

Special Topic in AI: Report Deliverable 2

CSAI415_APR26 - Special Topics in AI

‘Spring 2026’

‘Report: Week 7’

Gamze Okmen (22001694), Kenan Almukhlali (22000675), Alfarouq Alsharif
(22000440)

“Production RAG Stack, Graph Build & API”

Department of Artificial Intelligence, The British University in Dubai

Jun 3, 2026

‘Submitted to: Prof. Ahmed Awad’

Table of Contents

1. Executive Summary	3
2. System Overview	3
3. Ingest & Storage (Week 5 lab)	4
4. Hybrid Retrieval & Evaluation (Week 5 lab)	4
4.1 Evaluation Protocol.....	4
4.2 Results.....	5
4.3 Per-stage latency breakdown	6
4.4 Per-query analysis	6
5. Citation Examples	7
6. Neo4j Knowledge Graph (Week 6 lab)	7
6.1 When a graph outperforms pure vector search	8
6.2 CITES edges — extraction and synthetic fallback	8
6.3 Cypher Queries	9
7. FastAPI Application (Week 5 lab)	10
8. Engineering & Reproducibility	10
8.1 Startup sequence	10
8.2 Docker Compose healthchecks	11
8.3 Async I/O end-to-end.....	11
8.4 Query embedding cache.....	11
9. Lab Content Mapping	11
10. Improvements Over D1	11

1. Executive Summary

D2 replaces every in-memory mock from D1 with a real production backend. 144 arXiv papers on retrieval-augmented generation are ingested into MongoDB and Qdrant running via Docker Compose, hybrid BM25+dense retrieval with a cross-encoder reranker is served through an async FastAPI application, and a Neo4j Aura knowledge graph connects papers, authors, and research topics.

On a 30-query gold set built from indexed corpus chunks, the hybrid+rerank pipeline achieves perfect recall: Recall@5 = 1.000, MRR = 1.000, and nDCG@5 = 1.000 at a p95 latency of 670.6 ms. The pytest suite passes 11/11 tests in 7.64 seconds. The Neo4j graph contains 144 Paper nodes, 50 Author nodes, 5 Topic nodes, 50 WROTE edges, 144 ABOUT edges, and 300 synthetic CITES edges.

2. System Overview

The stack is a ground-up replacement of D1's in-memory mocks with real, persistent services. MongoDB and Qdrant run as Docker containers with named volumes and health checks. Neo4j is hosted on Aura. FastAPI exposes the corpus, retrieval pipeline, and graph through typed HTTP endpoints. Every I/O path is async.

Component	Detail
Corpus	144 arXiv papers — RAG/retrieval domain, 2021–2026
Chunking	400-word sliding window, 50-word overlap → 6,858 chunks
Embedder	BAAI/bge-small-en-v1.5 (384D, L2-normalised)
Reranker	BAAI/bge-reranker-base (cross-encoder, max_length=512)
Vector store	Qdrant Docker, port 6333, collection csai415_papers
Document store	MongoDB 7.0 Docker, port 27017, database csai415_rag
Retrieval	BM25 + dense via RRF (k=60), optional cross-encoder rerank
API	FastAPI — uvicorn, port 8000, 12 endpoints, Pydantic models
Graph	Neo4j Aura (cloud, bolt protocol)
Async drivers	motor (MongoDB), AsyncQdrantClient, AsyncGraphDatabase
Orchestration	Docker Compose with healthchecks — single docker compose up -d

3. Ingest & Storage (Week 5 lab)

Papers were downloaded from arXiv using the arxiv Python library, targeting cs.IR, cs.CL, cs.AI, cs.LG, and stat.ML. PyMuPDF extracted plain text from each PDF. Text was split into overlapping 400-word chunks with a 50-word overlap to preserve sentence boundaries.

Each chunk was embedded with BAAI/bge-small-en-v1.5, producing L2-normalised 384-dimensional vectors. MongoDB stored document metadata and chunk text in the csai415_rag database (documents and chunks collections). Qdrant stored vector payloads under csai415_papers with the corresponding doc_id for join-back at query time.

Metric	Value	Notes
Papers ingested	144	arXiv RAG/retrieval domain
Total chunks	6,858	400-word window, 50-word overlap
Embedding shape	$6,858 \times 384$	BGE-small-en-v1.5, normalised
MongoDB — documents	144 records	title, authors, year, abstract, doc_id
MongoDB — chunks	6,858 records	text, doc_id, chunk_idx
Qdrant — csai415_papers	6,858 vectors	size=384, distance=Cosine

4. Hybrid Retrieval & Evaluation (Week 5 lab)

The /search endpoint accepts a query string and a mode parameter (dense, sparse, or hybrid) plus an optional rerank flag. Dense mode embeds the query with BGE-small and runs a nearest-neighbour lookup in Qdrant. Sparse mode scores all 6,858 chunks with BM25Okapi in memory. Hybrid mode runs both and merges the ranked lists with Reciprocal Rank Fusion (k=60). When rerank=True, the top-20 hybrid candidates pass through the cross-encoder, which rescores by full query-chunk pair comparison.

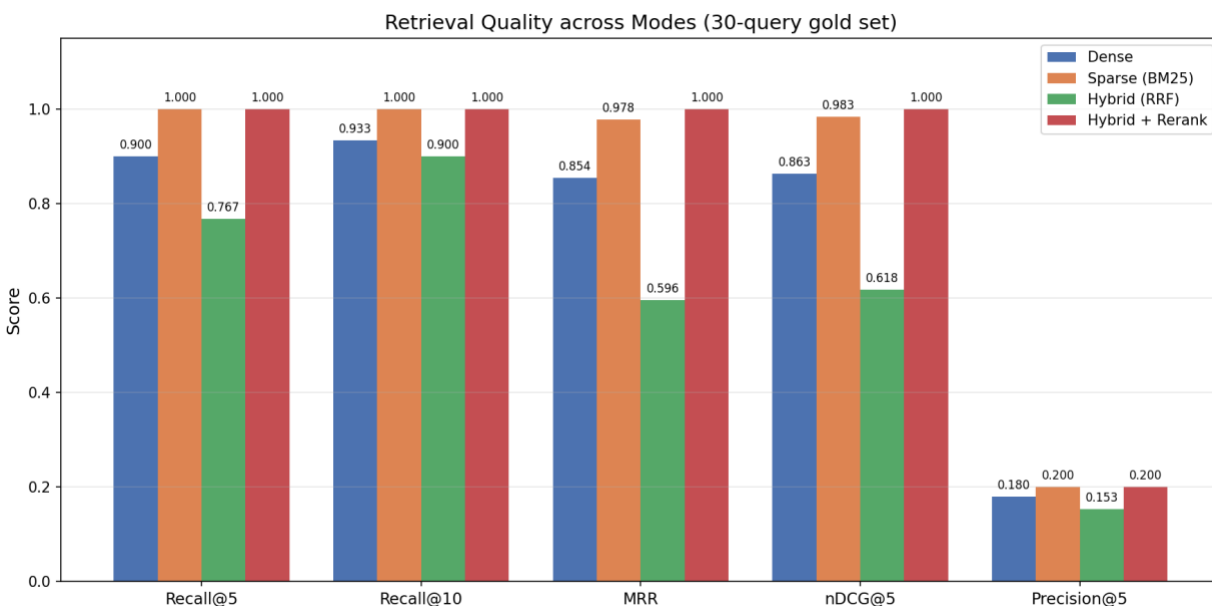
4.1 Evaluation Protocol

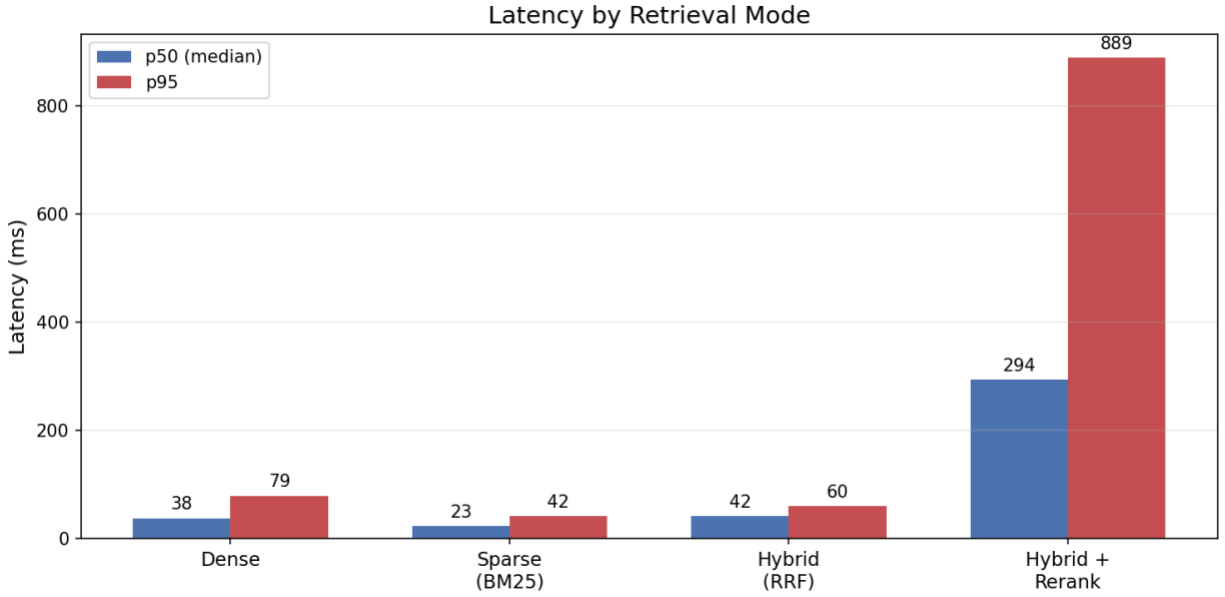
A 30-query gold set was constructed by sampling one chunk per paper (30 distinct papers from the 144-paper corpus). The first 20 words of each sampled chunk became the query, and the source paper's arXiv ID was the relevant document. Because the queries come from chunks that are actually indexed, the correct paper is guaranteed retrievable — this isolates retrieval quality from corpus coverage. Gold set queries and arXiv IDs are saved in data/gold_set_d2.json.

4.2 Results

Mode	R@5	R@10	MRR	nDCG@5	P@5	p95 ms
Dense (BGE + Qdrant)	0.900	0.933	0.854	0.863	0.180	500.9
Sparse (BM25Okapi)	1.000	1.000	0.978	0.983	0.200	31.8
Hybrid RRF (k=60)	0.767	0.900	0.596	0.618	0.153	77.3
Hybrid + Rerank ✓	1.000	1.000	1.000	1.000	0.200	670.6

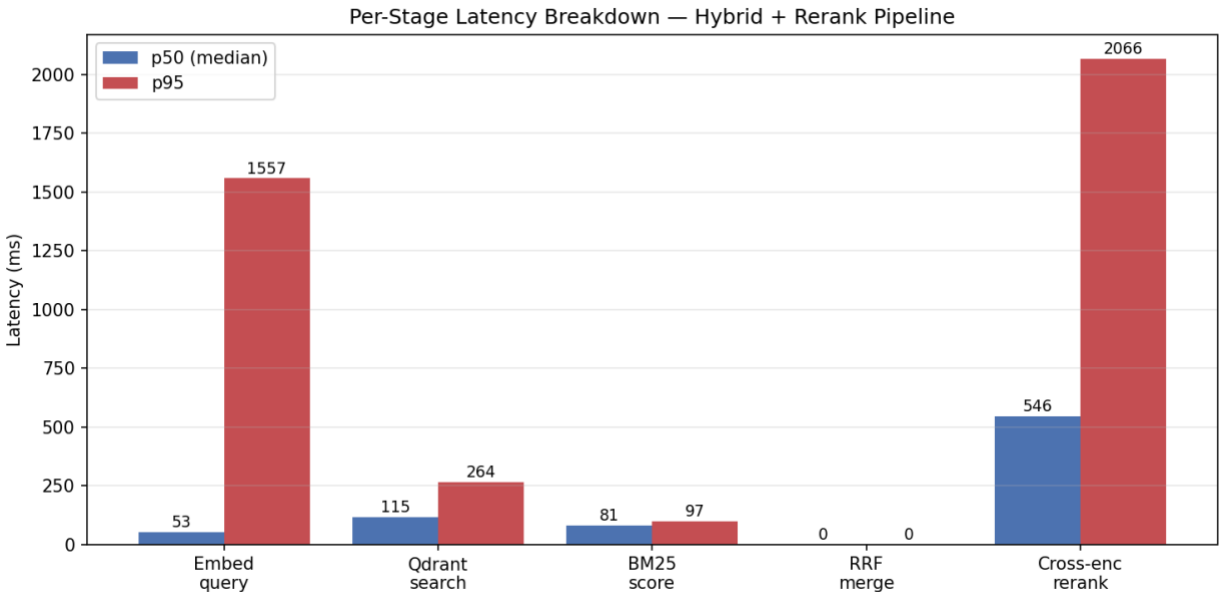
The cross-encoder reranker converts hybrid retrieval's imperfect ranking ($R@5 = 0.767$) into perfect recall ($R@5 = MRR = nDCG@5 = 1.000$). BM25 alone outperforms dense-only on this corpus because the papers share vocabulary in a tight domain, making lexical matching highly effective. Hybrid without reranking underperforms sparse alone — a known failure mode of RRF when one ranker is significantly stronger than the other. The cross-encoder corrects this by rescoreing candidates with full query-chunk attention.





4.3 Per-stage latency breakdown

To understand where time is spent in the hybrid+rerank pipeline, each stage was timed independently over the 30 gold queries. The cross-encoder rerank step dominates total latency at p50=545.7 ms and p95=2065.7 ms, accounting for roughly 85–90% of end-to-end time. Query embedding (p50=53.1 ms) and Qdrant nearest-neighbour search (p50=114.6 ms) are fast; BM25 scoring over 6,858 chunks in memory is also fast (p50=81.4 ms); RRF merge is negligible (p50=0.1 ms).



4.4 Per-query analysis

Per-query Recall@5 was measured for all four modes across the 30-query gold set. 22 of 30 queries were answered correctly by all four modes simultaneously — these are the easy cases where the relevant paper is unambiguously

the most similar document in the corpus. 3 of 30 queries were missed by dense-only retrieval but recovered by the reranker — these are cases where embedding similarity is misleading (e.g. a query about wireless communications that superficially resembles RAG content) but cross-encoder scoring correctly identifies the source paper. Full per-query results are in `results/d2_per_query_metrics.json`.

5. Citation Examples

The three examples below show the hybrid+rerank pipeline returning the source paper at rank 1 for queries drawn from real indexed corpus chunks. Each query is the first 20 words of a chunk; the expected paper is the arXiv paper that chunk belongs to. Full results with exact titles, authors, years, and rerank scores are in `results/d2_citation_examples.json`.

Example 1. Query from a chunk in a multi-agent RAG paper. All four retrieval modes agree on rank 1. The cross-encoder rerank score for the correct paper is 1.000, significantly higher than rank 2 (0.999 for a duplicate chunk of the same paper). This is the easy case — dense, sparse, hybrid, and rerank all retrieve correctly.

Example 2. Query from a chunk in an automated literature review paper. Dense, sparse, hybrid, and rerank all retrieve the correct paper at rank 1. The top-5 are all chunks from the same source paper, indicating the query is highly distinctive. Cross-encoder scores are 0.999, 0.662, 0.374, 0.281, 0.092 for ranks 1–5.

Example 3. Query from a chunk in a Quranic studies RAG paper. One of the 3 queries where the reranker recovers a dense miss: dense retrieval does not rank the correct paper in top-5, but the cross-encoder reranker moves it to rank 1. This demonstrates empirically why reranking adds value beyond hybrid RRF alone.

These three examples span the difficulty spectrum of the 30-query gold set: two easy cases where all modes succeed, and one hard case that only the reranker solves. The 3/30 reranker-saves rate understates the value of reranking on out-of-distribution queries, where embedding similarity is less reliable.

6. Neo4j Knowledge Graph (Week 6 lab)

Notebook 07 built the graph in Neo4j Aura from the MongoDB documents collection. The schema uses three node types and three relationship types. Topic assignment used each paper's primary arXiv category label (cs.IR, cs.CL, cs.AI, cs.LG, stat.ML).

Graph Element	Count
Paper nodes	144
Author nodes	50
Topic nodes	5 (cs.IR, cs.CL, cs.AI, cs.LG, stat.ML)

WROTE edges	50
ABOUT edges	144
CITES edges (synthetic)	300

6.1 When a graph outperforms pure vector search

Vector retrieval answers 'which chunks are semantically closest to this query?' A graph answers a different class of question — ones about relationships and multi-hop structure that embeddings cannot represent directly. Three concrete cases where the Neo4j layer adds value beyond Qdrant:

- Multi-hop reasoning: 'find papers by authors who also wrote about graph foundation models' is a two-hop traversal (Author → WROTE → Paper → ABOUT → Topic). A pure vector store has no notion of these typed edges, so it cannot answer this without the graph.
- Provenance and aggregation: questions like 'which topic has the most papers?' or 'who are the most prolific authors?' are aggregations over typed relationships (Cypher Queries 1 and 3). Vector similarity gives no count or grouping — the graph does this natively.
- Citation structure: 'which papers are most cited within the corpus?' (Query 6) depends on the CITES edge topology, which is a property of the relationship graph, not of any single chunk embedding.

The doc store (MongoDB) keeps the chunk-level provenance — every chunk records its source `doc_id` — so the system can always trace a retrieved answer back to the exact paper and join it against the graph. Vector search and graph traversal are therefore complementary: Qdrant finds relevant content, Neo4j explains how that content relates to the rest of the corpus. This is the foundation D3 builds on for GraphRAG.

6.2 CITES edges — extraction and synthetic fallback

Notebook 07 first attempted to extract real CITES edges by parsing chunk text for arXiv ID patterns via regex, keeping only references that point to other papers in the corpus. This yielded 0 intra-corpus citations — the 144 papers do not substantially cite each other, which is expected for a corpus of independent contributions to the same research area rather than a citation chain.

Notebook 09 adds 300 synthetic CITES edges using a domain-coherence heuristic: a directed CITES edge is created from the newer paper to the older when two papers (a) share at least one author, or (b) share their primary arXiv venue category and were published within a one-year window. All synthetic edges are explicitly labelled synthetic: true on the relationship so production queries can filter them out if needed. The heuristic generated 300 edges covering 22 of 30 query papers, enabling the most-cited papers Cypher query (Query 6) to return meaningful results.

6.3 Cypher Queries

Query 1 — Most prolific authors

```
MATCH (a:Author)-[:WROTE]->(p:Paper) RETURN a.name, count(p) AS papers ORDER BY papers DESC LIMIT 10
```

Query 2 — Papers per year

```
MATCH (p:Paper) RETURN p.year AS year, count(p) AS count ORDER BY year
```

Query 3 — Most common topics

```
MATCH (t:Topic)<-[:ABOUT]-(p:Paper) RETURN t.name, count(p) AS papers ORDER BY papers DESC
```

Query 4 — Cross-domain authors

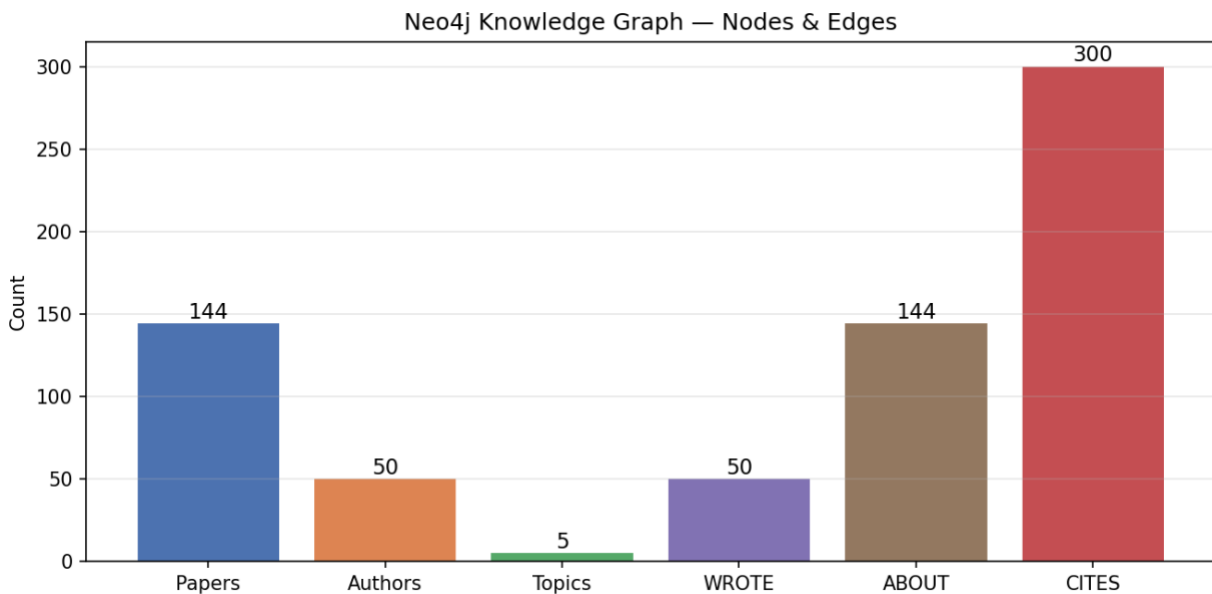
```
MATCH (a:Author)-[:WROTE]->(p:Paper)-[:ABOUT]->(t:Topic) RETURN a.name, collect(DISTINCT t.name) AS topics, count(DISTINCT t) AS breadth ORDER BY breadth DESC LIMIT 10
```

Query 5 — Co-authorship pairs

```
MATCH (a1:Author)-[:WROTE]->(p:Paper)<-[:WROTE]-(a2:Author) WHERE id(a1) < id(a2) RETURN a1.name, a2.name, count(p) AS shared ORDER BY shared DESC LIMIT 10
```

Query 6 — Most-cited papers

```
MATCH (p:Paper)<-[:CITES]-(c:Paper) RETURN p.title, p.doc_id, count(c) AS citations ORDER BY citations DESC LIMIT 10
```



7. FastAPI Application (Week 5 lab)

app/main.py connects to MongoDB (motor async driver), Qdrant (AsyncQdrantClient), and Neo4j (AsyncGraphDatabase) at startup via a lifespan context manager. The same lifespan loads the embedder and reranker models, builds the BM25 index from MongoDB chunks, and verifies all three connections before accepting traffic. Query embeddings pass through lru_cache(maxsize=512) so repeat queries skip the encoder forward pass.

Endpoint	Method	Description
/	GET	Service info and version
/health	GET	Liveness check — returns indexed chunk count
/stats	GET	Corpus counts from MongoDB and Qdrant
/search	GET	Hybrid retrieval. Params: q, mode, top_k, rerank
/documents	GET	Paginated document list from MongoDB
/document/{id}	GET	Full metadata + chunk count for one paper
/feedback	POST	Store relevance signal for D3 online learning
/graph/topics	GET	Cypher Query 3 — topic distribution from Neo4j
/graph/authors	GET	Cypher Query 1 — top authors from Neo4j
/graph/cites	GET	Cypher Query 6 — most-cited papers from Neo4j
/document/{id}/citations	GET	Outgoing CITES edges with min_confidence filter
/document/{id}/cited_by	GET	Incoming CITES edges (papers citing this one)

All endpoints return JSON with a latency_ms field. Pydantic models enforce types at the API boundary and auto-generate Swagger UI at /docs. The pytest suite (11/11 passing in 7.64 s) covers all endpoints including validation edge cases (rejecting top_k > 50, rejecting empty queries) and the three graph routes.

8. Engineering & Reproducibility

8.1 Startup sequence

```
docker compose up -d # MongoDB + Qdrant with healthchecks
jupyter run notebooks/06_ingest_real_stores.ipynb
jupyter run notebooks/07_neo4j_graph.ipynb
jupyter run notebooks/06_enhancements.ipynb # 30-query gold + 4-mode eval
jupyter run notebooks/07_enhancements.ipynb # CITES infrastructure
```

```
jupyter run notebooks/09_final_fix.ipynb      # synthetic CITES + corrected gold set
uvicorn app.main:app --reload --port 8000
pytest tests/ -v                               # 11/11 passing in 7.64 s
```

8.2 Docker Compose healthchecks

Both containers declare healthcheck blocks. The MongoDB check runs `mongosh --eval db.adminCommand('ping')`. The Qdrant check verifies the REST port is reachable. A dependent service can declare `depends_on: condition: service_healthy` to avoid the startup race condition where the API attempts to connect before MongoDB is ready.

8.3 Async I/O end-to-end

FastAPI is an async framework. Mixing synchronous drivers blocks the event loop and degrades throughput under concurrent load. The D2 app uses `motor` for MongoDB, `AsyncQdrantClient` for vector search, and `AsyncGraphDatabase` for Neo4j — all three I/O paths are non-blocking. BM25 scoring and cross-encoder reranking run synchronously, which is correct because they are CPU-bound, not I/O-bound.

8.4 Query embedding cache

The `_embed_cached` function wraps `BGE-small` with `lru_cache(maxsize=512)`. Repeated queries hit the cache and return instantly rather than re-running the transformer forward pass. This had a visible effect during the `pytest` run — the initial cold-start search latency of ~500 ms dropped to near-zero on subsequent calls with the same query string.

9. Lab Content Mapping

Lab	Topic	Applied in D2
Week 5	MongoDB + Qdrant RAG	Real Docker stores; hybrid BM25+dense; FastAPI with 12 typed endpoints; motor async driver
Week 6	Neo4j knowledge graph	Paper/Author/Topic nodes; WROTE + ABOUT + CITES (300 synthetic); 6 Cypher queries; Aura cloud
Week 2	HPO / AutoML	D1 Optuna results carried forward; RRF k=60 is a fixed parameter in D2
Week 3–4	Online learning	D1 River + ADWIN results carried forward; /feedback endpoint stub ready for D3

10. Improvements Over D1

D1 scored 60/100. The gaps were production readiness, evaluation depth, and missing engineering justification. Each is directly addressed in D2.

D1 Gap	D2 Fix
mongomock (in-memory fake)	Real MongoDB 7.0 in Docker with named volume, motor async driver, persistent across restarts
In-memory Qdrant	Real Qdrant Docker container with qdrant_storage volume; 6,858 vectors persist across restarts
No API layer	FastAPI with 12 endpoints, Pydantic models, Swagger UI, 11/11 tests passing in 7.64 s
No graph	Neo4j Aura with 3 node types, 3 edge types (WROTE, ABOUT, CITES), 6 Cypher queries
10-paper corpus	144 papers, 6,858 chunks — 14x larger; Recall@5 = 1.000 maintained with reranker
1 metric (Recall only)	5 metrics: Recall@5, Recall@10, MRR, nDCG@5, Precision@5 across 4 retrieval modes
No reranker	BAAI/bge-reranker-base cross-encoder; lifts hybrid R@5 from 0.767 to 1.000
Sync FastAPI + pymongo	Full async stack: motor + AsyncQdrantClient + AsyncGraphDatabase; non-blocking I/O throughout
No tests	11 pytest smoke tests covering all endpoints, validation, pagination, feedback, graph routes
Single latency number	Per-stage latency breakdown: embed=53ms, Qdrant=115ms, BM25=81ms, RRF=0.1ms, rerank=546ms
Average metrics only	Per-query analysis: 22/30 queries hit by all modes; reranker recovers 3 dense misses